Detecting LLM-Written Code on AIGCodeSet: Tuned Code Encoders versus N-gram Baselines

Andreas Bartsiokas
LT2326 Machine learning for statistical NLP: Advanced
November 2026

Code: github.com/mparts/CODer
Data: huggingface.co/datasets/mparts/CODer-aigcodeset
Models: huggingface.co/mparts/CODer-unixcoder

Abstract

In this project I look at whether code written by an LLM can be told apart from human-written code, and which LLM wrote it, using the AIGCodeSet dataset. I compare word and character n-gram logistic regression with fine-tuned code encoders (CodeBERT, GraphCodeBERT, UniXcoder) on four task variants, with splits grouped by problem and a test set that was used only once. On dev, a tuned character n-gram baseline does better than CodeBERT and GraphCodeBERT. A tuned UniXcoder beats the baseline on all four variants, but the difference is statistically clear only for binary detection. Averaging six seeds and mixing the result with the baseline improves the scores a bit more. Most errors come from two places: code that an LLM *edited* from a human submission (detected only 27–54% of the time, against 94–98% for code written from scratch), and Gemini, whose from-scratch code is often classified as human. In an ablation of the character baseline, identifier names turn out to be the strongest cue.

1 Introduction

Telling LLM-generated code apart from code written by students or competitive programmers is hard to do just by looking at it. This has been in the spotlight for the last couple of years: professors worry about students handing in AI-written work, and in industry many companies adopt AI tools while others enforce strict no-AI policies. I approach the problem empirically, with three questions: (i) How well do cheap lexical baselines and fine-tuned code encoders do on the task? (ii) Which hyperparameters and cues actually matter? (iii) Where do the models fail? I do not try to replicate the dataset paper (Demirok and Kutlu, 2025), which reports binary detection with classical classifiers on a random split, so my numbers cannot be compared with theirs directly. Recent work also shows that detection results depend a lot on the evaluation setting, and that they get worse under more realistic conditions, such as unseen generators, code written partly by humans and partly by AI, edited code, and distribution shift (Orel et al., 2026; Demirok et al., 2026).

2 Data

AIGCodeSet (Demirok and Kutlu, 2025) pairs human solutions from CodeNet (Puri et al., 2021) with code from three LLMs (CodeLlama 34B, Codestral 22B and Gemini 1.5 Flash). The LLM code comes in two kinds: either the LLM writes a solution from scratch (*generate*), or it *edits* a human submission to fix a runtime error or a wrong output. This difference matters, because both AIGCodeSet and MultiAIGCD find that AI-edited code is much harder to detect than code generated from scratch (Demirok and Kutlu, 2025; Demirok et al., 2026). After deduplication I keep 7,531 samples and split them by problem into train (5,267), dev (739) and test (1,525), so no problem appears in more than one split. Some files have identical code under both labels (478 of them), and I remove those from every split, which leaves 4,953 train, 681 dev and 1,419 test files. Removing them makes the splits slightly easier than the raw data.

From this I build four task variants: `binary` (human vs AI) and `multiclass` (human, GEMINI, LLAMA, CODESTRAL) use all the data, while `binary_generate` and `multiclass_generate` drop the edited code. Table 1 shows the test set sizes. The models only ever see a normalized copy of the code, with line endings unified, tabs expanded to spaces, and trailing spaces, leading blank space and final-newline differences removed. I did this because an audit of the raw files had found formatting quirks that give the source away (for example, 21% of LLAMA files start with whitespace). Obviously, no metadata column is given to the models.

Variant	Human	AI (per class)
binary / multiclass	905	514 (GEMINI 179, LLAMA 146, CODESTRAL 189)
binary_generate / multiclass_generate	905	186 (GEMINI 62, LLAMA 61, CODESTRAL 63)

Table 1: Test set sizes.

3 Methods

Baselines. My baselines are TF-IDF features with logistic regression, on either word or character n-grams, with balanced class weights. For the character model I picked n-grams (1,6) and C=10 with 7-fold cross-validation on the training folds only, capping the features at 300k. The word model just reuses C=10.

Encoders. I fully fine-tune three pretrained code encoders: CodeBERT (Feng et al., 2020), GraphCodeBERT (Guo et al., 2021) and UniXcoder (Guo et al., 2022). All of them use AdamW with a linear schedule and 10% warmup, weight decay 0.01, batch size 16 and learning rate 2e-5, and train for 4 epochs without class weights. CodeBERT and GraphCodeBERT run at 128 tokens with the default dropout of 0.1, and I did not tune them further. Since half of the files are longer than 128 tokens, the final UniXcoder reads the first 256 tokens instead (in a two-seed test this gave about +.01, short of the tuning rule below, Appendix C), and it uses dropout 0.0 (§4.2). I keep the checkpoint with the best dev macro-F1 at the end of an epoch. All models are trained on the train split only.

Tuning protocol. I tuned on the two generate variants with seeds 0–2. A change was kept only if it raised the mean dev score by at least .01 on both variants, and at most one of the six paired seed comparisons went the other way. The dev set is small (only 93 AI files in `binary_generate` and 31 per class in `multiclass_generate`), so I treat differences below about .03 between single runs as ties. The final models use fresh seeds 3–8, and each run is evaluated on test only once.

Ensembles and statistics. The *seed ensemble* averages the class probabilities of the six UniXcoder runs. The *mix* then averages the ensemble with the character baseline at a weight of 0.5. I did not choose this weight on dev (I only looked at 0.3 and 0.7 in some exploratory runs). To test the gap to the baseline I use a paired bootstrap over problems (1,000 resamples), so that files from the same problem stay together. The seeds are held fixed, so the intervals do not include variation between seeds. All scores are macro-F1.

4 Results

4.1 Final test results

UniXcoder beats the character baseline on all four variants (Table 2, per seed in Appendix B and ROC curves in Appendix D). However, a single model is significantly better only on `binary`. The seed ensemble is significantly better on three of the four, although only barely on `multiclass` and `binary_generate`. The mix scores highest everywhere, but I only added it after seeing the dev results (where it was tied with the ensemble), so I report it as an extra column without an interval. On dev, the fresh seeds score close to the tuning seeds (.923 and .818 against .926 and .829 on the generate variants), so the tuning result holds up on new seeds.

4.2 What mattered in training

At 128 tokens on dev, the choice of pretrained checkpoint made the biggest difference: UniXcoder scored .896 / .778, GraphCodeBERT .869 / .739 and CodeBERT .834 / .712 (`binary_generate` / `multiclass_generate`), and UniXcoder beat GraphCodeBERT in all six paired seed comparisons. CodeBERT and GraphCodeBERT were never evaluated on test. Inverse-frequency class weights hurt the multiclass task (CodeBERT on `multiclass`: .404 with weights against .515 without), because they flooded the human-heavy dev set with false AI predictions.

Out of all the changes in Table 3, only removing dropout gave a consistent gain across all six paired seeds. On `binary_generate`, the training loss drops close to zero while dev F1 barely moves after the second epoch (Figure 1). On the small multiclass dev set, dev F1 still moves by .02–.04 between neighbouring epochs (Figure 5).

4.3 Which cues carry the signal

To see which cues the character baseline relies on, I ran an ablation on its input, removing the layout or the comments, or masking the identifier names

Variant	char LogReg	word LogReg	UniXcoder, 1 seed	Seed ensemble	Mix 0.5	1 seed − char	Ensemble − char
binary	.743	.658	.772 ± .002	.778	.783	+.029 [+.011, +.048]	+.034 [+.015, +.054]
multiclass	.554	.477	.571 ± .026	.595	.614	+.017 [−.016, +.052]	+.041 [+.003, +.077]
binary_generate	.923	.885	.939 ± .008	.943	.948	+.016 [−.003, +.035]	+.020 [+.001, +.041]
multiclass_generate	.808	.772	.837 ± .016	.839	.867	+.029 [−.006, +.066]	+.031 [−.008, +.070]

Table 2: Test macro-F1. UniXcoder single seed is mean ± std over 6 seeds. The intervals are 95% bootstrap intervals for the difference from the character baseline.

Setting	binary_generate	multiclass_generate
dropout 0.1 (starting point)	.906	.798
dropout 0.0	**.926**	**.829**
dropout 0.2	.862	.751
dropout 0.0, 6 epochs	.930	.815
dropout 0.0, batch 8 / 32	.924 / .921	.819 / .803
dropout 0.0, weight decay 0.1	.921	.821
dropout 0.1: 384 tokens / lr 3e-5 / lr 1e-5 (6 epochs)	.911 / .904 / .901	.774 / .793 / .780

Table 3: UniXcoder, 256 tokens, dev macro-F1, seeds 0–2. Appendix A lists all tuning runs.

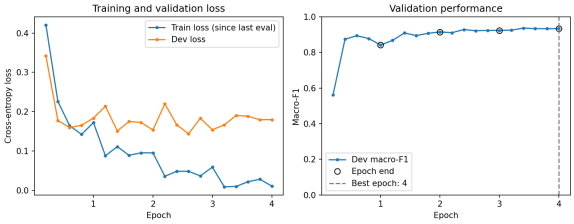


Figure 1: Training curves of UniXcoder, seed 5, `binary_generate`, dev macro-F1 and loss, 5 evaluations per epoch.

Input	binary_ generate	multiclass_ generate
full code	.909	.791
layout removed	.907	.778
comments removed	.902	.787
identifier names masked	.882	.765

Table 4: Macro-F1 mean over folds.

(7-fold CV on the training folds, generate variants only).

Masking identifiers costs the most (about .026, Table 4), although that is only about one standard deviation across folds. Many of the top n-grams of the full model are pieces of names, like `_`, `get_` and `min_` (Figure 2, more in Appendix H). Still, most of the signal survives name masking, so none of the three cues explains detection on its own. I did not check whether the encoders use the same cues.

4.4 Where the models fail

Edited code is the main source of error (Table 5). It makes up 328 of the 514 AI files in `binary`, and it is detected far less often than code written

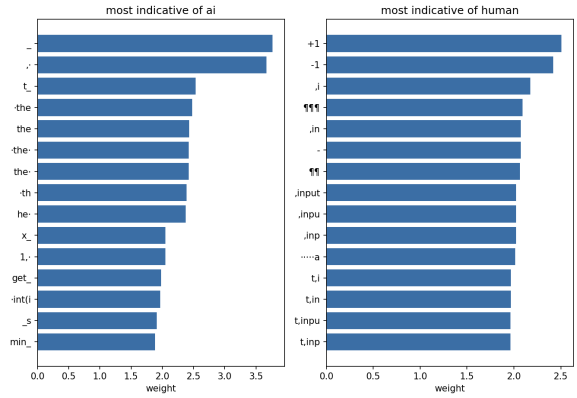


Figure 2: Top character n-grams of the char LogReg, `binary_generate`. · is a space, ¶ a newline.

from scratch, which explains most of the gap between `binary` (.78) and `binary_generate` (.94). In `multiclass_generate`, the ensemble predicts Human for 37% of the GEMINI files, but another LLM for only 8% (LLAMA 2%, CODESTRAL 6%). So when the models get Gemini wrong, they mostly take it for human code. Figure 3 shows this for a single seed (more in Appendices F and G). When the code is written from scratch, telling the LLMs apart mostly works: F1 is .91 (LLAMA), .82 (CODESTRAL) and .64 (GEMINI) for the ensemble, and .96, .86 and .67 for the mix. Finally, on the full data about 1 in 10 human files gets flagged as AI.

5 Discussion

Supported by my results. (1) A tuned character n-gram model is a strong baseline, and CodeBERT and GraphCodeBERT do not reach it on dev un-

True source	n	UniXcoder ensemble	char LogReg
Human (false positives)	905	9.9%	8.7%
LLAMA, generate	61	98.4%	98.4%
CODESTRAL, generate	63	98.4%	92.1%
GEMINI, generate	62	93.5%	90.3%
LLAMA, edit	85	54.1%	43.5%
CODESTRAL, edit	126	54.0%	35.7%
GEMINI, edit	117	27.4%	22.2%

Table 5: Share of files detected as AI, by true source, `binary`, test.

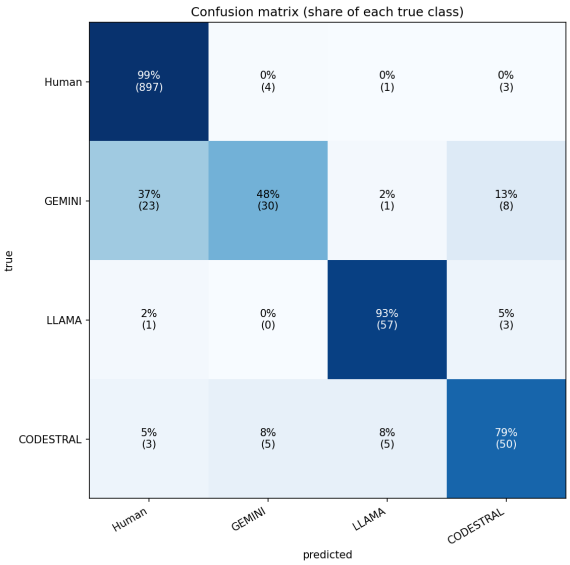


Figure 3: Confusion matrix, UniXcoder seed 3, `multiclass_generate`, test.

der a problem-grouped split. (2) UniXcoder without dropout is at least as good on every variant, and clearly better on `binary`. (3) Averaging seeds gives a small but reliable gain, and combining the encoder with the n-gram model gave the best test scores. (4) Edited code and Gemini are the hardest cases, as they were in the AIGCodeSet paper (Demirok and Kutlu, 2025).

Guesses I did not test. UniXcoder might do better than CodeBERT because of its pretraining objectives. Dropout may hurt because the training set is small. The mix may help because the two models make different mistakes. And Gemini may simply write code that is closer to human style.

6 Conclusion

On AIGCodeSet, a tuned UniXcoder (dropout 0.0, 256 tokens) is ahead of the best n-gram baseline on every variant, with a clear margin on binary detection and smaller, mostly non-significant margins on the rest. Seed averaging and mixing with the baseline add a little on top. The hard cases are still edited code and Gemini. The most useful next steps would be an external human-only test set, to check false positives, and a split that holds out one LLM, to test how well the models generalize.

Limitations

- The dev set is small, so tuning decisions on the multiclass variants are noisy. I compared about 20 configurations on dev, so I may have overfit to it. The fresh seeds score only .003–.011 lower there, which rules out lucky seeds.
- `binary` and `multiclass` reuse a configuration that was tuned on the generate variants only.
- The test sets are small for the LLM classes (61–63 files each in the generate variants), so several intervals include zero.
- I did not keep the cross-validation and ablation code, so Table 4 cannot be re-run.
- I added the mix after seeing dev, and I did not tune its weight.
- Everything comes from a single data source: there is no out-of-distribution evaluation and no external human-only test set (such as MBPP).
- I didn't run a larger code LLM with LoRA.
- I only use one dataset and one programming language, and recent benchmarks show that detector performance can vary a lot across languages, generators and evaluation scenarios (Orel et al., 2026; Demirok et al., 2026).

References

Basak Demirok and Mucahid Kutlu. 2025. AIGCodeSet: A new annotated dataset for AI generated code detection. *Preprint*, arXiv:2412.16594.

Basak Demirok, Mucahid Kutlu, and Selin Mergen. 2026. MultiAIGCD: A comprehensive dataset for AI generated code detection covering multiple languages, models, prompts, and scenarios. *Neural Computing and Applications*, 38(16).

Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. 2020. CodeBERT: A pre-trained model for programming and

natural languages. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 1536–1547, Online. Association for Computational Linguistics.

Daya Guo, Shuai Lu, Nan Duan, Yanlin Wang, Ming Zhou, and Jian Yin. 2022. UniXcoder: Unified cross-modal pre-training for code representation. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 7212–7225, Dublin, Ireland. Association for Computational Linguistics.

Daya Guo, Shuo Ren, Shuai Lu, Zhangyin Feng, Duyu Tang, Shujie Liu, Long Zhou, Nan Duan, Alexey Svyatkovskiy, Shengyu Fu, Michele Tufano, Shao Kun Deng, Colin Clement, Dawn Drain, Neel Sundaresan, Jian Yin, Daxin Jiang, and Ming Zhou. 2021. GraphCodeBERT: Pre-training code representations with data flow. In *International Conference on Learning Representations (ICLR)*.

Daniil Orel, Dilshod Azizov, Indraneil Paul, Yuxia Wang, Iryna Gurevych, and Preslav Nakov. 2026. AICD Bench: A challenging benchmark for AI-generated code detection. In *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 6913–6938, Rabat, Morocco. Association for Computational Linguistics.

Ruchir Puri, David S. Kung, Geert Janssen, Wei Zhang, Giacomo Domeniconi, Vladimir Zolotov, Julian Dolby, Jie Chen, Mihir Choudhury, Lindsey Decker, Veronika Thost, Luca Buratti, Saurabh Pujar, Shyam Ramji, Ulrich Finkler, Susan Malaika, and Frederick Reiss. 2021. CodeNet: A large-scale AI for code dataset for learning a diversity of coding tasks. *Preprint*, arXiv:2105.12655.

A All tuning runs

Table 6 lists every tuning run, including the CodeBERT and GraphCodeBERT runs and the 128-token runs on `binary` and `multiclass`, which Table 3 leaves out. Only the 128-token runs cover all four variants. Everything at 256 tokens and above was tuned on the generate variants only (§3).

Model	Len	Ep	LR	BS	WD	Drop	binary	multiclass	binary_ generate	multiclass_ generate
char LogReg	–	–	–	–	–	–	.739	.568	.911	.794
word LogReg	–	–	–	–	–	–	.672	.455	.856	.688
CodeBERT	128	4	2e-5	16	0.01	0.1	.724 ± .009	.512 ± .011	.834 ± .009	.712 ± .028
CodeBERT†	256	4	2e-5	16	0.01	0.1	–	–	.853 ± .028	.726 ± .001
GraphCodeBERT	128	4	2e-5	16	0.01	0.1	–	–	.869 ± .012	.739 ± .015
UniXcoder	128	4	2e-5	16	0.01	0.1	.749 ± .005	.541 ± .011	.896 ± .005	.778 ± .004
UniXcoder	256	4	2e-5	16	0.01	0.1	–	–	.906 ± .010	.798 ± .025
UniXcoder	256	6	2e-5	16	0.01	0.1	–	–	.907 ± .007	.816 ± .016
UniXcoder	256	4	3e-5	16	0.01	0.1	–	–	.904 ± .008	.793 ± .011
UniXcoder	256	6	1e-5	16	0.01	0.1	–	–	.901 ± .016	.780 ± .016
UniXcoder	384	4	2e-5	16	0.01	0.1	–	–	.911 ± .008	.774 ± .009
UniXcoder	256	4	2e-5	16	0.01	0.2	–	–	.862 ± .007	.751 ± .016
UniXcoder	**256**	**4**	**2e-5**	**16**	**0.01**	**0.0**	–	–	**.926 ± .005**	**.829 ± .023**
UniXcoder	256	6	2e-5	16	0.01	0.0	–	–	.930 ± .004	.815 ± .019
UniXcoder	256	4	2e-5	8	0.01	0.0	–	–	.924 ± .014	.819 ± .008
UniXcoder	256	4	2e-5	32	0.01	0.0	–	–	.921 ± .011	.803 ± .015
UniXcoder	256	4	2e-5	16	0.1	0.0	–	–	.921 ± .014	.821 ± .010

Table 6: All tuning runs, dev macro-F1 at the best epoch, mean ± std over seeds 0–2 (†seeds 0–1 only). Len: tokens read, Ep: epochs, LR: learning rate, BS: batch size, WD: weight decay, Drop: dropout. All encoders use AdamW with a linear schedule, 10% warmup and no class weights. The baselines are fit once. –: not run. The final UniXcoder configuration is in bold.

B Results per seed

Table 7 lists the six final UniXcoder runs behind Table 2. On `binary` the seeds are within .007 of each other on test, but on `multiclass` they go from .534 (seed 6) to .607 (seed 8).

		Seed						
Variant	**Split**	**3**	**4**	**5**	**6**	**7**	**8**	**Mean ± std**
`binary`	dev	.757	.763	.757	.767	.755	.764	.760 ± .005
	test	.774	.773	.769	.771	.776	.772	.772 ± .002
`multiclass`	dev	.546	.580	.567	.548	.576	.583	.567 ± .016
	test	.554	.588	.574	.534	.570	.607	.571 ± .026
`binary_generate`	dev	.928	.917	.934	.914	.925	.918	.923 ± .007
	test	.947	.932	.949	.937	.942	.930	.939 ± .008
`multiclass_generate`	dev	.810	.807	.849	.826	.819	.794	.818 ± .019
	test	.819	.846	.825	.861	.829	.841	.837 ± .016

Table 7: Macro-F1 of the final UniXcoder runs, per seed. The test means are the single-seed column of Table 2.

C Input length

The encoders only read the start of each file. With the UniXcoder tokenizer, the median training file is 128 tokens long, 90% of the files are shorter than about 365 tokens, and the longest one has 4,360. 49.8% of the files are longer than 128 tokens, 20.7% longer than 256, 9.1% longer than 384 and 5.1% longer than 512. Human and AI files are about the same length (median 129 and 123 tokens), so cutting them does not give the label away. I did not keep the code for this count.

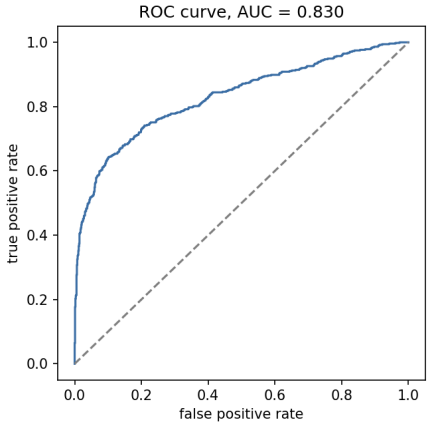
Table 8 compares 256 and 128 tokens on the same seeds. For UniXcoder, 256 tokens gave about +.01 on both generate variants, which is just short of the tuning rule (§3). Seed 2 at 256 tokens was only run later, as the starting point of Table 3, and gives +.007 and +.039. Reading 384 tokens did not help (Table 6).

	UniXcoder		CodeBERT	
Seed	**binary_ generate**	**multiclass_ generate**	**binary_ generate**	**multiclass_ generate**
0	+.004	−.003	+.049	+.044
1	+.018	+.023	−.006	+.009
Mean	+.011	+.010	+.022	+.027

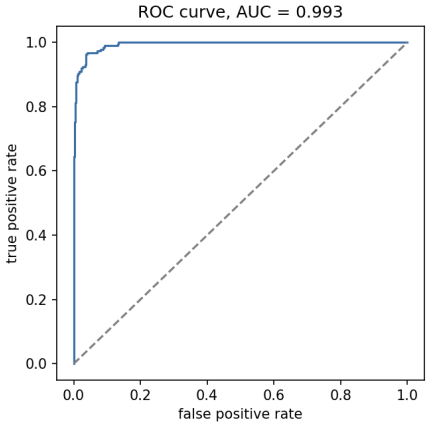
Table 8: Dev macro-F1 at 256 tokens minus 128 tokens, same seed, dropout 0.1.

D ROC curves

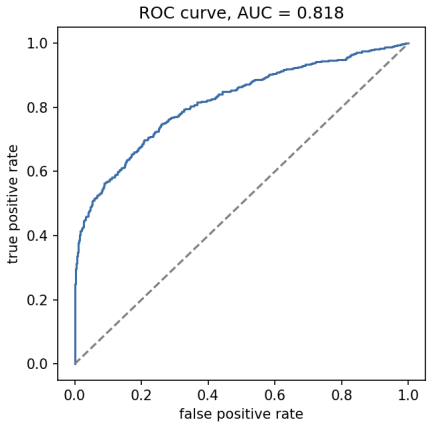
Figure 4 shows ROC curves on test for the binary variants, with AI as the positive class. In this and the following appendices, the single UniXcoder runs are seed 3, as in Figure 3, and the training curves are seed 5, as in Figure 1. Seed 3 is close to the mean on `binary` (.774 against .772), but below it on `multiclass` (.554 against .571, Table 7). None of the figures show the seed ensemble or the mix.



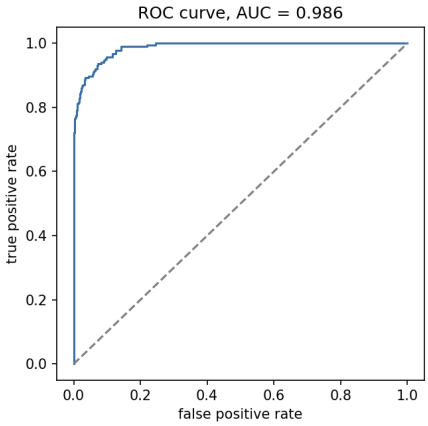
(a) UniXcoder, `binary`



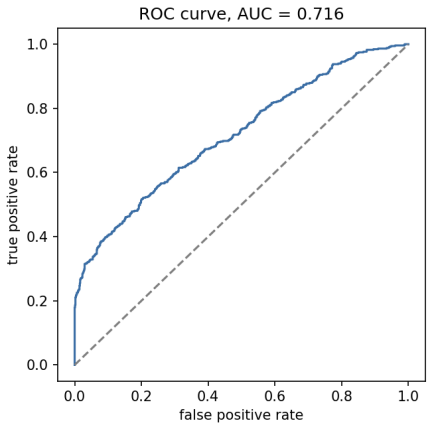
(b) UniXcoder, `binary_generate`



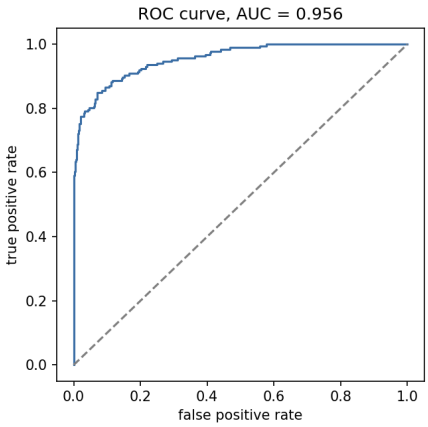
(c) char LogReg, `binary`



(d) char LogReg, `binary_generate`



(e) word LogReg, `binary`

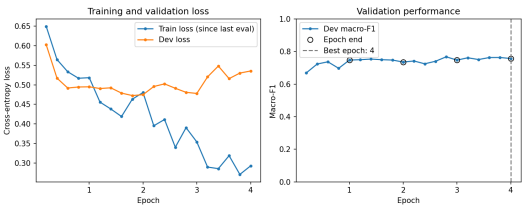


(f) word LogReg, `binary_generate`

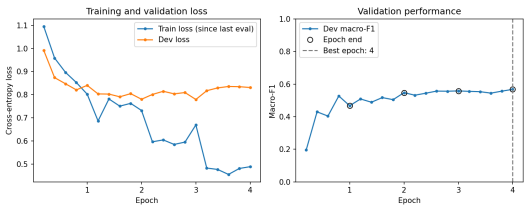
Figure 4: ROC curves on test, with AI as the positive class. The AUC is in each plot's title.

E Training curves

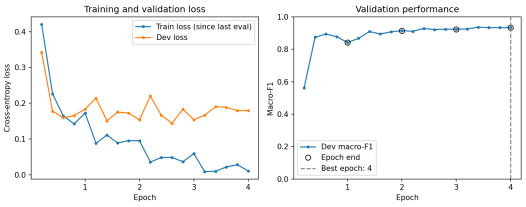
Figure 5 shows the training curves of UniXcoder on dev, for all four variants.



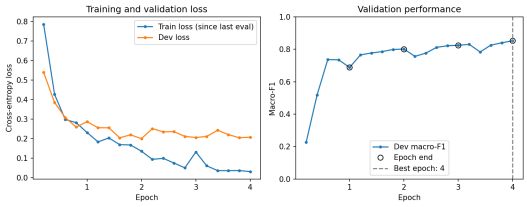
(a) `binary`



(b) `multiclass`



(c) `binary_generate`

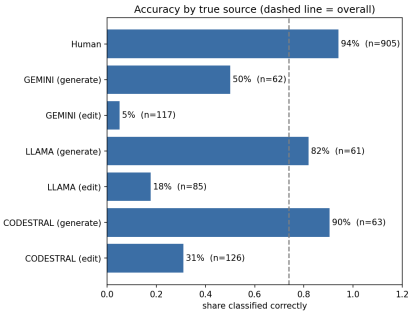


(d) `multiclass_generate`

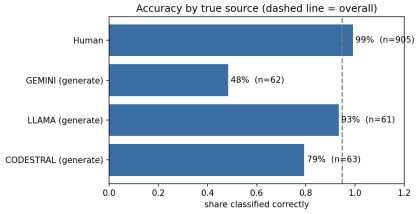
Figure 5: Training curves of UniXcoder, seed 5, dev macro-F1 and loss, 5 evaluations per epoch. Panel (c) is Figure 1.

F Accuracy by source

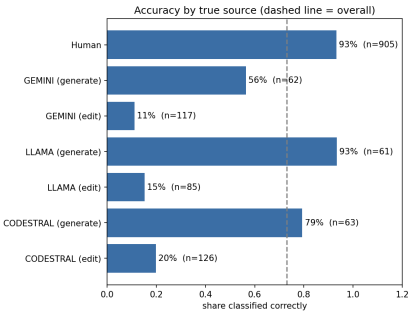
Figure 6 shows the share of test files from each source that get the right class, on the multiclass variants. In `multiclass`, the edited files rarely get the right LLM: UniXcoder gets 5% of the edited GEMINI files right, against 50% of the generated ones.



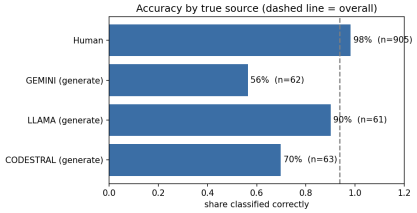
(a) UniXcoder, `multiclass`



(b) UniXcoder, `multiclass_generate`



(c) char LogReg, `multiclass`

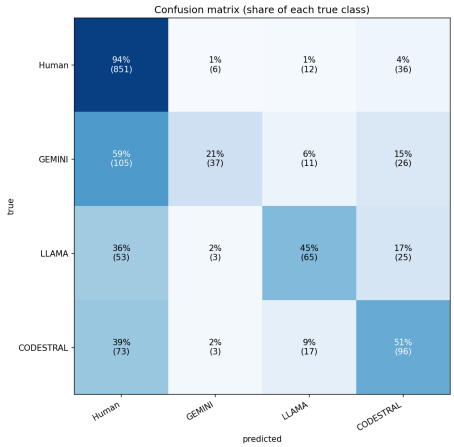


(d) char LogReg, `multiclass_generate`

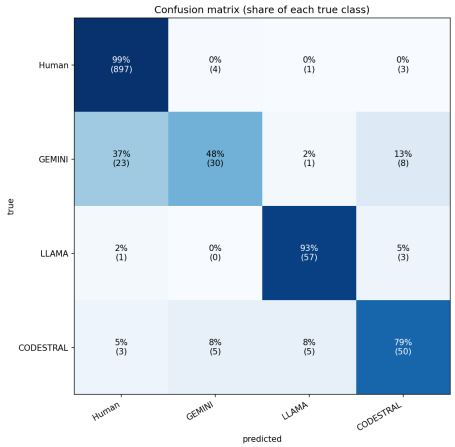
Figure 6: Share of test files classified correctly, by true source, on the multiclass variants. The dashed line is the overall accuracy.

G Confusion matrices

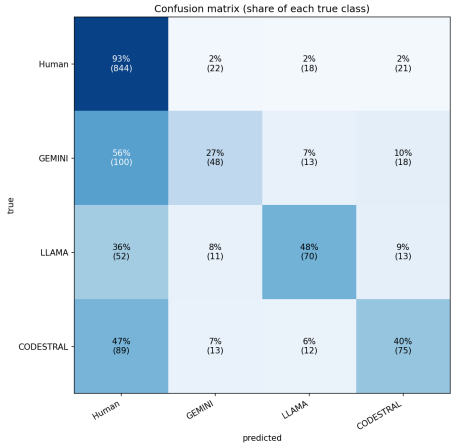
Figure 7 shows the confusion matrices of all three models on the multiclass variants, on test.



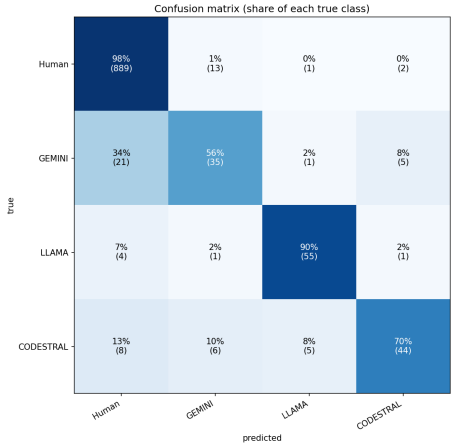
(a) UniXcoder, `multiclass`



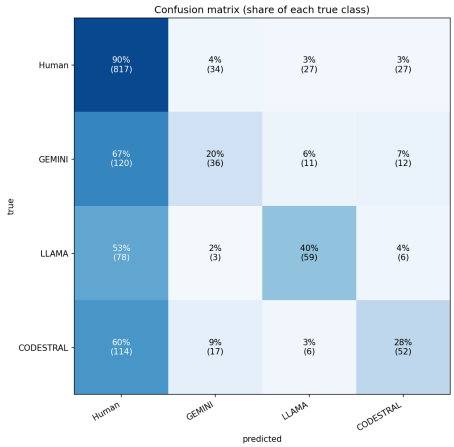
(b) UniXcoder, `multiclass_generate`



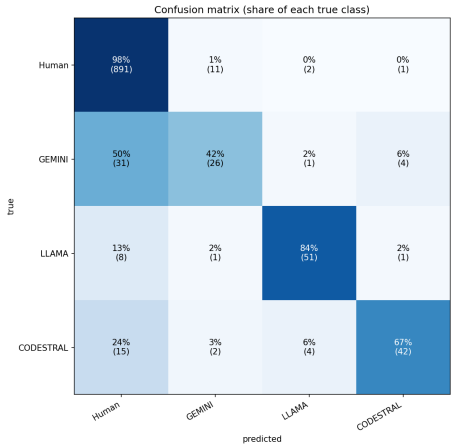
(c) char LogReg, `multiclass`



(d) char LogReg, `multiclass_generate`



(e) word LogReg, `multiclass`

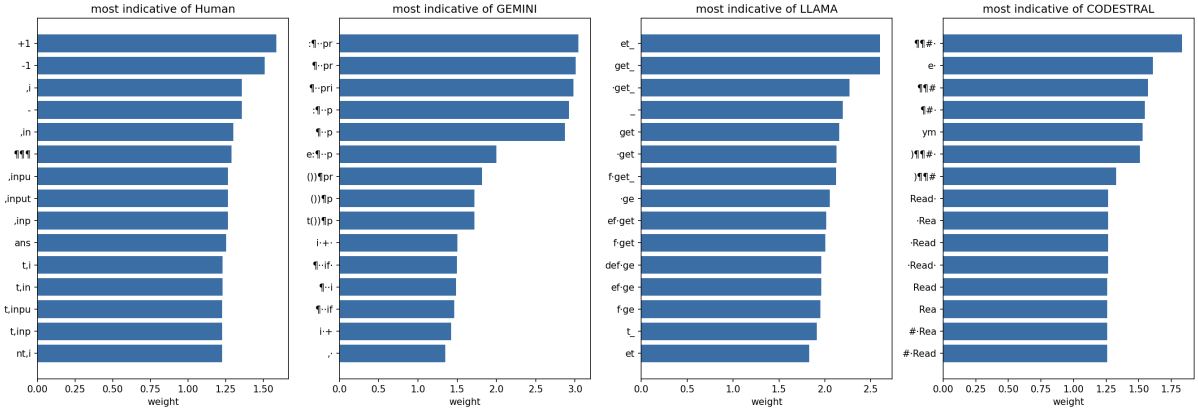


(f) word LogReg, `multiclass_generate`

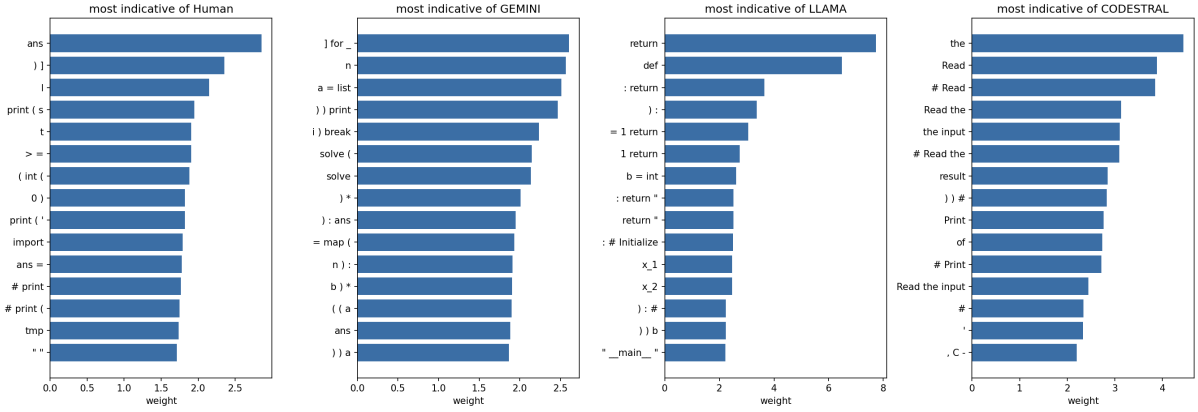
Figure 7: Confusion matrices on test, as the share of each true class with the count in parentheses. Panel (b) is Figure 3.

H Top n-grams

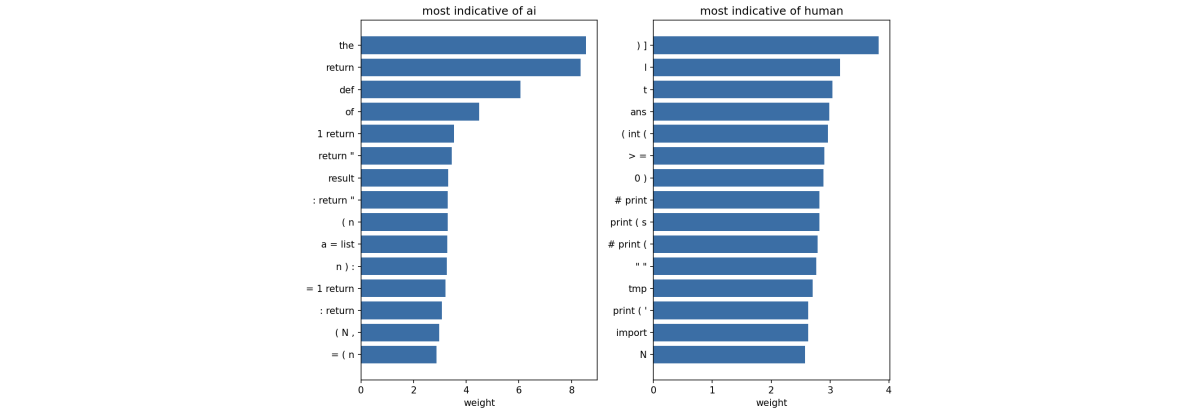
Figure 8 shows more top n-grams: both baselines on `multiclass_generate`, and the word model on `binary_generate`.



(a) char LogReg, `multiclass_generate`



(b) word LogReg, `multiclass_generate`



(c) word LogReg, `binary_generate`

Figure 8: Top n-grams of the LogReg baselines. · is a space, ¶ a newline. Figure 2 shows the char model on `binary_generate`.